

Neural Taint Tracking for Prompt-Injection-Resistant Agents

Project Idea and Initial Progress (Report 1)

Albina Akhmetova

1 Project Idea: the Taint-Aware Orchestrator

The project implements a *Taint-Aware* orchestrator to mitigate Indirect Prompt Injection in multi-agent systems. We build a hybrid neural-symbolic taint-tracking mechanism that propagates trust labels from data sources to privileged tool invocations.

The system utilizes three distinct roles:

- **Researcher Agent:** Fetches external RAG/Web content. All results are stamped `untrusted` by construction.
- **Sanitizer Agent:** The sole neural component (LLM-based) responsible for declassifying untrusted strings by invoking a specialized `gate.sanitize` tool.
- **Executive Agent:** Authorized to call *privileged tools* (filesystem, shell), provided the inputs satisfy the non-interference constraint.

2 What Has Been Done (Initial Progress)

- **Environment.** `uv sync` completed; `uv run taicf smoke` passes the three stub configs over the lite suites (filesystem / memory / retrieval), producing per-case spans, violations, and reports under `runs/`. No LLM is involved, so the baseline is deterministic and re-scorable.
- **Code audit.** Analysed `src/scaffolds/langgraph_react.py` and identified `_to_lc_tool` (verified by source inspection) as the `StructuredTool` wrapper and primary instrumentation point for injecting trust metadata into OTel spans. Studied `src/evaluation/store_jsonl.py` to confirm how span attributes are persisted and re-loaded for post-hoc scoring.
- **Design Specification:** Defined the `taicf.trust.*` metadata namespace for span attributes and drafted the logic for a lineage-based Taint-Scanner.

3 Theoretical Formulation

Threat Model Card. Research claim is the tuple $\text{claim} = (O, B, J, C, E)$ shipped with code and a query log. For the Taint-Aware orchestrator:

O **Oracle** — what one interaction returns to the attacker: whether an instruction injected through untrusted retrieval/memory produces an observable privileged side-effect or tool output.

B **Budget** — poisoned chunks/documents, tokens per payload, agent turns, tool-call retries, and the detection threshold of the scanner.

J **Objective** — attacker: drive the Executive Agent to call a privileged tool with attacker-controlled arguments (tool-argument hijacking, exfiltration). Defender (dual): minimise unsafe privileged-action success.

C **Constraint** — the attack reaches the agent only through untrusted channels (web/RAG/memory), is semantic natural language, and must survive the sanitizer; the defender’s constraint is non-interference on privileged actions.

- **E Evidence** — the per-claim evidence set is:
 - $\text{ASR}_{\text{priv}}@B$ — attack success rate on privileged actions at budget *B* (primary security metric);
 - *Clean-case utility* — task-success rate on benign cases (e.g. `fs-01-clean`); the utility control that prevents a trivially-blocking defence from scoring well;
 - *Over-block rate* — fraction of benign privileged calls wrongly blocked (false-positive control for the Taint-Scanner);
 - taint-leakage violation count from the Taint-Scanner;
 - the OTel trace, sanitizer declassification logs, and a re-scorable reproducibility bundle.

We additionally name the protected **Asset** = {privileged tools: filesystem, shell, network} and the attacker **Capability** = control of untrusted retrieval/memory content but *not* the trusted system prompt or the sanitizer. (Asset/Capability are stated explicitly because the agentic setting needs them; *O* and *B* in the card already encode the attacker’s interface and effort.)

Formal Objective: We minimize the Attack Success Rate while enforcing information-theoretic non-interference ($I = 0$):

$$\min_{\theta} \mathbb{E}[ASR_{priv}(\theta)] \quad \text{s.t.} \quad I(A_{priv}; X_u \mid X_t, S) \leq \epsilon \quad (1)$$

Zero **taint_leakage** violations across the suite provide a sound certificate of non-interference, assuming robust taint propagation.

4 Practical Formulation

The proposed system implements separation of concerns through automated metadata propagation. The **Researcher** stamps every read result with `taicf.trust.level=untrusted` on tools tagged **retrieve** or **memory**. Because the TAICF Projection layer admits only whitelisted attributes onto each **Step**, we will extend its implementation to propagate the `taicf.trust.*` namespace from raw spans to the **Trace**; this ensures the scanner can observe the trust state.

The **Sanitizer** is implemented as an agent role (the sole neural component) that, upon successful declassification of data, calls a specialized `gate.sanitize` tool. This invocation emits a `taicf.kind=TOOL` span with `trust.level=sanitized`. Finally, the **Taint-Scanner** is implemented as a pure deterministic predicate over the **ScanContext** (`needs_truth = False`). Attached to **PRE_TOOL** and **POST_TURN** hooks, it resolves the provenance of privileged arguments and emits a **taint_leakage** violation if an untrusted ancestor reaches a privileged tool without a covering `gate.sanitize` event in its lineage. Mathematically, zero violations across the suite serve as a sound certificate of non-interference ($I(A_{priv}; X_u \mid X_t, S) = 0$), contingent on the soundness of the taint propagation mechanism.

Value-based taint propagation. Provenance is resolved by *value-based* taint tracking. Every tool span records its output string; outputs of **retrieve/memory** tools enter an untrusted taint set T_u , while a `gate.sanitize` span removes the substrings it declassifies from T_u . For each privileged call A_{priv} , the scanner normalises every argument string and tests it for any surviving substring drawn from T_u ; a match with no covering `gate.sanitize` event in its lineage raises a **taint_leakage** violation. This is more precise than a temporal over-approximation for text agents, at the cost of scan-time overhead and some conservative over-blocking (hence the over-block rate is reported as a control).

Soundness of the certificate. If the suite yields zero violations, then no untrusted output reaches a privileged argument verbatim without declassification. Under the *lexical-flow assumption*—that the agent does not transform untrusted content before placing it in A_{priv} —the privileged action is a function of (X_t, S) alone, giving $I(A_{priv}; X_u \mid X_t, S) = 0$. The residual gap is semantic laundering (the LLM paraphrases or re-encodes the injected payload, defeating substring matching); we treat this as an explicit threat and surface it in the failure-trace analysis (S6) rather than claiming unconditional soundness.

5 Implementation Roadmap (S1–S6)

Stage	Course Gate	Milestone	Exit Criterion / Evidence
S1	Claim design (1–3)	Claim Card fixed	(O, B, J, C, E) + Asset/Capability + taint YAML schema fixed.
S2	Pilot evidence (4–6)	Instrumentation	Trust-tagging via <code>_to_lc_tool</code> ; Projection extended to carry <code>taicf.trust.*</code> .
S3	Pilot evidence (4–6)	Taint-Scanner	Value-based deterministic scanner in <code>builtins.py</code> ; unit-tested on hand-built ScanContext .
S4	Pilot evidence (4–6)	Adversarial + Sanitizer	AgentDojo-style dev split; <code>gate.sanitize</code> integration.
S5a	Artifact freeze (7–8)	Freeze	<code>config_hash</code> pinned; spans + scanners frozen.
S5b	Hidden/Fresh (9–10)	Survival runs	$ASR_{priv}@B$ on held-out injections; clean-case utility + over-block rate reported.
S6	Cross-review (11–14)	Paper + Re-score	4-page report incl. failure-trace analysis; independent taicf score reproduces verdicts.

Table 1: Implementation roadmap mapped to the course delivery gates.